HotelPro

Full Stack Project Documentation

A practical guide to the application architecture, data model, workflows, setup, and deployment.

Repository: https://github.com/ChhaviJain24/HotelPro-Project

Project Description

HotelPro is a web-based hotel management system. Guests can search rooms, place reservations, pay, and manage their stays. Front-desk staff can handle arrivals, walk-ins, room readiness, and service tasks. Administrators manage room inventory, rates, users, hotel settings, payments, refunds, and operational reports. The application is built as a React single-page client connected to an Express REST API and a MongoDB database.

Scenario

A guest selects dates and a room, reviews the nightly price and tax, and creates a short-lived pending reservation. Payment changes the reservation to confirmed. At the property, staff check the guest in and out; check-out creates a housekeeping task and makes the room available again after cleaning is marked complete. Guests can request help during an active stay and review a completed stay. Administrators oversee room inventory, staff accounts, revenue, occupancy, and refund requests.

Project at a glance

Area	Implementation
Application	Hotel booking and property operations web application
Architecture	React SPA → Express REST API → MongoDB
Client	React 18, Vite, Redux Toolkit, React Router, Axios, Bootstrap 5
Server	Node.js, Express 4, Mongoose, JWT, bcryptjs
Additional services	Razorpay integration option, Nodemailer, Multer image uploads
Primary roles	Guest, staff, administrator

System Requirements

The repository documents the following development prerequisites. Production deployments also need managed database, API, and client hosting accounts.

Software requirements

Operating System: Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.

Node.js (v16 or above): Provides the runtime environment for building and managing frontend logic and Powers the server-side logic and handles API routing. 👉 Download Node.js

npm (v8 or above): A package manager required to install dependencies for React and related libraries.

React.js: A JavaScript library for building dynamic and responsive user interfaces.

Browser: Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.

Express.js: A lightweight web framework for building RESTful APIs.

MongoDB: A NoSQL database used to store structured and unstructured data related to patient,Admin, doctor 👉 Download MongoDB

Postman: Tool for testing APIs during development.

Visual Studio Code: Preferred code editor with built-in Git and terminal support.

Git & GitHub: For version control and collaborative development.

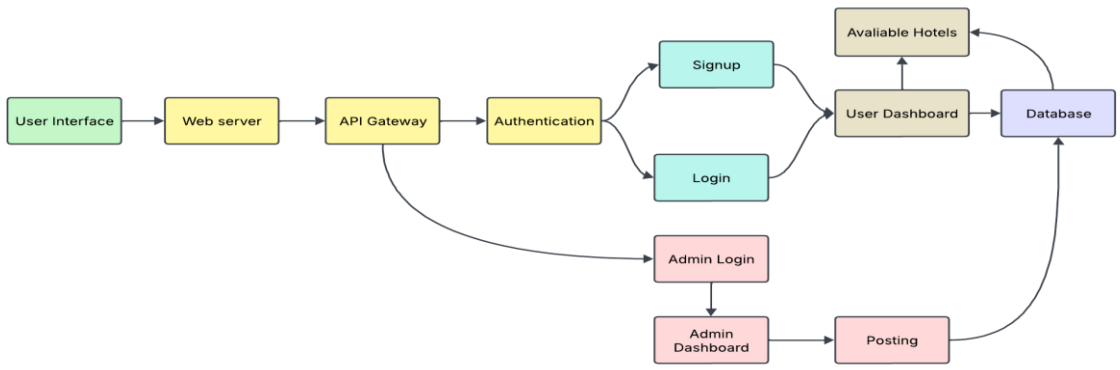
Hardware baseline

Describes the minimum and recommended specifications needed to support the development and usage of the application.
Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
RAM: Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
Storage: At least 1 GB free space — Required for package installations, MongoDB setup, and local project files.

Technical Architecture

Guest / Staff / Admin	**React + Vite client**	**Express REST API**	**MongoDB**

The client handles navigation, role-aware screens, and API requests. The API validates requests, applies authentication and role checks, runs booking and payment rules, and reads or writes MongoDB through Mongoose models. Optional integrations send mail through SMTP, accept uploaded room images on the server, and connect to Razorpay when configured.



In this technical architecture , HotelPro consist of several components :

User Interface: The user interface provides options of User to signup/Login.

Web server: Hosts the user interface and serves the web pages to the users.

API Gateway: Acts as a Double entry point for User and admin requests and routes them to the respective services.

Authentication : The signup for the User and the user data will be stored in the respective collection.

Database: it uses three different collections for Rooms and bookings.

Request path

Layer	Responsibility
React interface	Pages, forms, room browsing, booking flow, staff console, admin console
API routes and middleware	HTTP endpoints, input checks, JWT session validation, role authorization, error handling
Business utilities and services	Availability overlap checks, tax/rate quote, notifications, email, payment gateway and refunds
Mongoose models	Schemas, field validation, references, password hashing hooks, persistence
MongoDB	Durable application records

The repository uses a route-and-model organization rather than a separate named controller directory. Express route modules coordinate request handling and business operations; Mongoose models define and persist the data. React supplies the presentation layer. This is a layered MVC-style design, adapted for a REST API and SPA.

Data Model and Relationships

The data layer is defined in server/models/index.js. MongoDB stores each entity as a collection; ObjectId references connect related records.

Entity	Key information	Relationships
User	Name, email, hashed password, role, active status, walk-in flag	Guests own bookings, payments, reviews, and notifications; staff/admin users create or update operational records.
Hotel	Name, contact details, check-in/out times, tax rate	Hotel configuration supplies tax and invoice details.
Room	Unique room number, category, price, seasonal prices, capacity, amenities, images, status, rating	Rooms are referenced by bookings, reviews, and service requests.
Booking	Guest, room, dates, guest count, quote, status, payment status, invoice, source	References one guest and one room; may have payment, review, and service activity.
Payment	Booking, user, amount, method, transaction ID, refund data	References a booking and user.
Review	Booking, room, user, 1–5 rating, comment	One review per booking; updates room average rating.
Notification	User, title, message, read flag	Belongs to a user.
ServiceRequest	Type, description, room, booking, requester, assignee, status	Can link to a room, booking, requester, and assigned staff member.
AdminLog	User, action, details	Records administrator actions.

Core guest journey

Browse rooms and choose dates	**Check availability and quote**	**Create pending booking**	**Pay and confirm**	**Stay, request service, check out, review**

Room availability is checked against overlapping confirmed or checked-in bookings and recent pending holds. A pending online booking holds dates for 15 minutes. The quote sums nightly rates, applies any matching seasonal prices, then adds the hotel tax rate (12% by default).

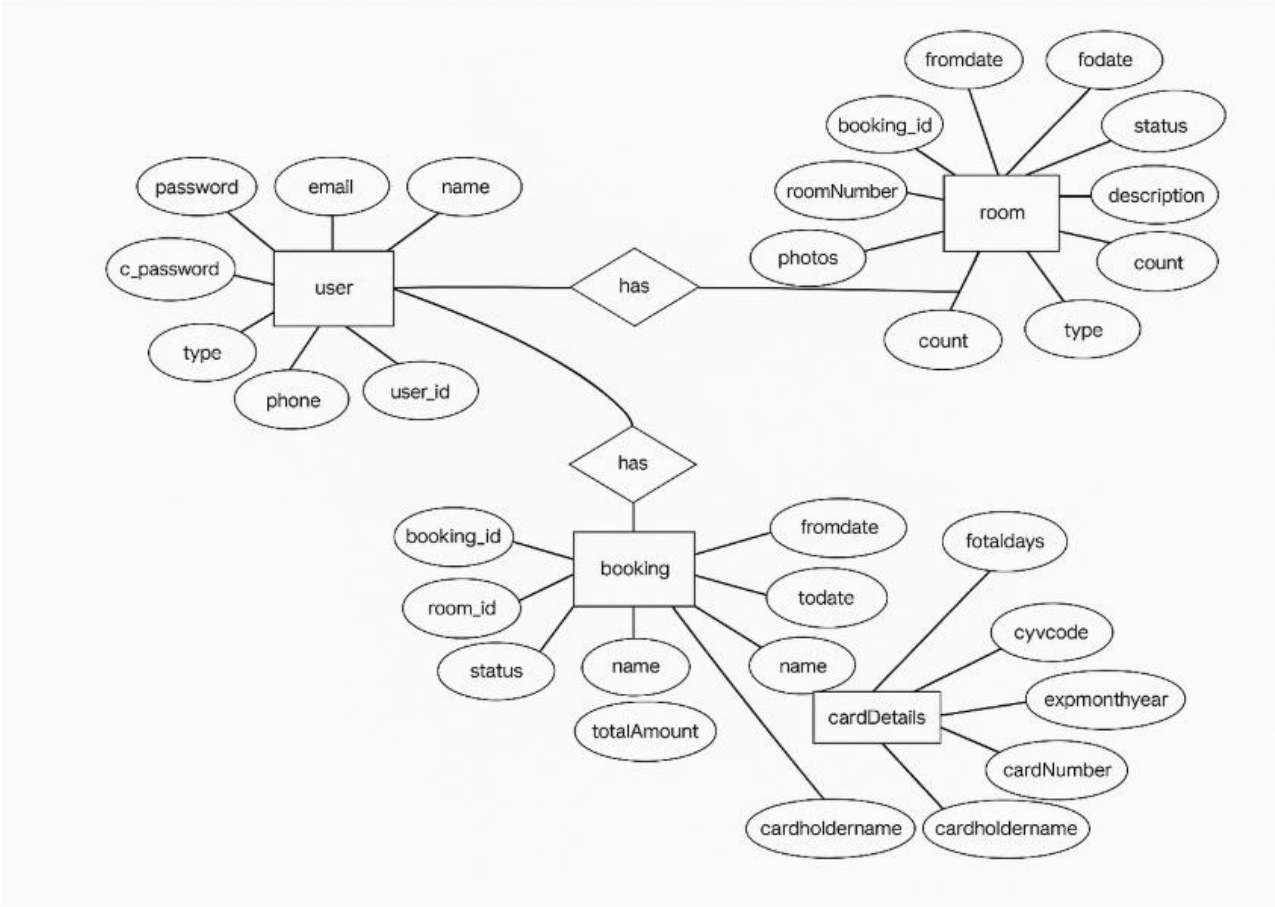
Front-desk journey

Find or enter a guest	**Choose dates and available room**	**Record cash, card, or UPI payment**	**Confirm or check in**	**Check out; complete housekeeping**

Booking and room states

Record	States represented in the code
Booking	Pending → Confirmed → Checked-in → Checked-out; can be Cancelled
Payment status	Unpaid, Paid, Refund-requested, Refunded
Room	Available, Occupied, Cleaning, Maintenance
Service request	Open, In progress, Done

ER Diagram



Features

Guest experience	Front desk operations	Administration
Register, sign in, reset password	View/filter reservations	Dashboard with revenue and occupancy metrics
Search by dates, guests, category, and price	Create walk-in bookings and record desk payment	Create/edit/delete rooms, prices, seasonal rates, and images
Room detail, amenities, reviews	Check guests in and out; view/print invoice	Manage staff and user access
Book, pay, cancel, request refund	Update room status and handle service tasks	Approve refunds and update bookings
View bookings, payments, notifications	Search returning guests	Set hotel details and tax; inspect admin action log
Submit stay review and service request	Complete housekeeping to release rooms	View payment records and reports

Roles and responsibilities

Role	Typical responsibilities	Access boundaries
Guest	Manage own account and bookings, pay, cancel, request services while checked in, review after check-out.	Protected guest routes restrict records to the signed-in guest.
Staff	Manage reservations, create walk-ins, check in/out, update room status, work service requests.	Staff routes allow operational tasks; admin-only settings remain restricted.
Administrator	Manage rooms, users, hotel settings, payments, refunds, reports, and action logs.	Admin authorization guards administrative API operations.

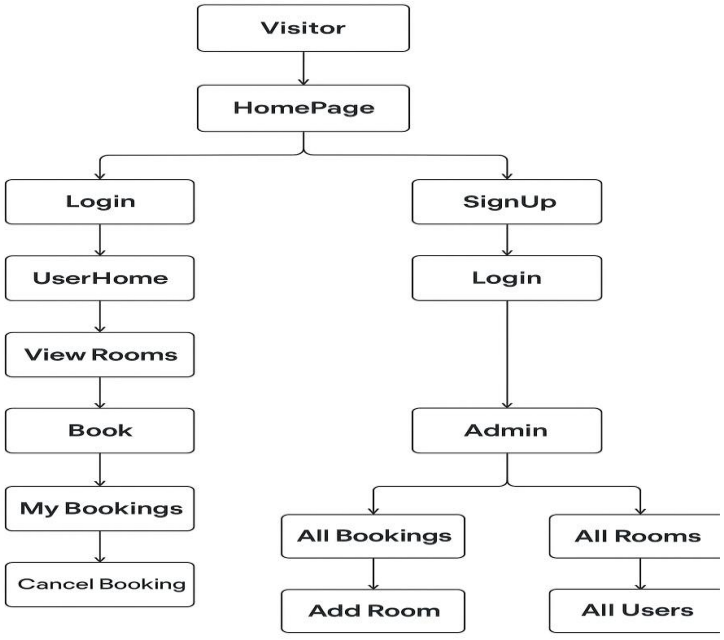
Authentication and access control

Passwords are hashed with bcryptjs before storage. The client sends a JWT bearer token for protected requests. Middleware loads the account, rejects disabled or expired sessions, and checks role permissions. Password reset uses a time-limited hashed token. These mechanisms are visible in the repository; a separate security audit was not performed for this document.

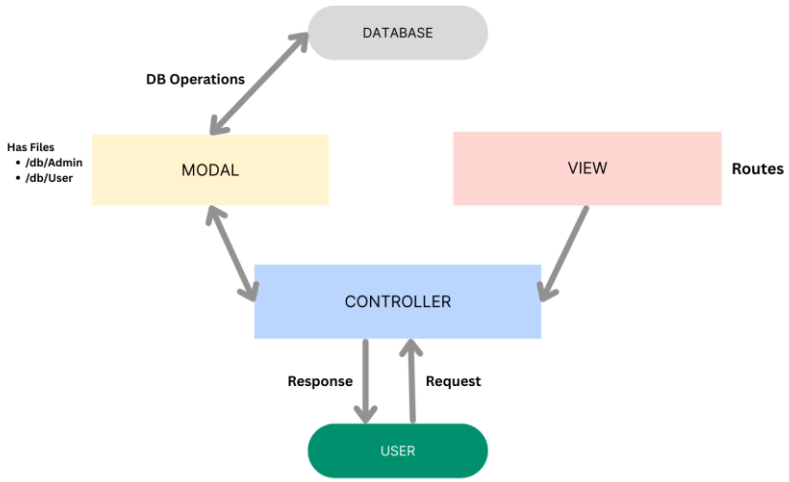
Payment behavior

PAYMENT_MODE=mock simulates a successful development payment and does not move real money. PAYMENT_MODE=razorpay enables order creation, signature verification, and a webhook flow when valid server keys and a public webhook URL are configured. The repository supports both paths; actual live readiness depends on merchant credentials and deployment configuration.

User Flow



MVC Pattern



Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB.

Controller Layer

The Controller layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

View Layer (Routing Layer)

In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Project Setup and Configuration

The repository is organized into client/ and server/. Start MongoDB and prepare environment configuration before running the application.

Install and start locally

1. Open one terminal in the server directory and install dependencies.
2. Configure the required server environment variables, including MONGO_URI and JWT_SECRET.
3. Seed demo data once if a fresh development database is desired. The seed script clears the collections it manages before creating users, hotel settings, and 12 rooms.
4. Start the API.
5. Open a second terminal in client/, install dependencies, and start the Vite development server.

```
cd server
npm install
npm run seed    # optional; clears seeded collections in the selected database
npm run dev     # API: http://localhost:5000

cd client
npm install
npm run dev     # client: http://localhost:5173
```

Environment configuration

Variable	Use
MONGO_URI	MongoDB connection string for the hotelpro_system database.
JWT_SECRET	Strong private key used to sign API sessions; required by server startup.
PORT	Optional API port; defaults to 5000.
CLIENT_URL	Allowed browser origin for CORS; local default is http://localhost:5173.
PAYMENT_MODE	mock for development or razorpay for configured online payments.
RAZORPAY_KEY_ID / RAZORPAY_KEY_SECRET	Server credentials for Razorpay order and signature operations.
RAZORPAY_WEBHOOK_SECRET	Secret used to verify Razorpay webhook signatures.
EMAIL_HOST, EMAIL_PORT, EMAIL_USER, EMAIL_PASS, EMAIL_FROM	SMTP transport and sender details for email delivery.
VITE_API_URL	Client API base URL for deployed builds.

Keep secrets in an untracked server/.env file or hosting-provider settings. Do not commit credentials. The repository deployment guide describes separate settings for Render and Netlify/Vercel.

Seeded demonstration accounts

The seed script creates these local demo accounts. Change or disable them before any public deployment.

Role	Email	Password
Administrator	admin@hotelpro.com	Admin@123
Staff	staff@hotelpro.com	Staff@123
Guest	guest@hotelpro.com	Guest@123

Backend and Database Development

The API entry point connects to MongoDB before listening. Express mounts route modules under /api, serves uploaded room images, exposes a health endpoint, and applies centralized not-found and error handlers. Protected routes use JWT authentication and role-based authorization.

Route group	Representative operations
/api/auth	Register, login, current account, password reset
/api/rooms	Search/detail; admin inventory and image management; staff room status
/api/bookings	Guest booking and cancellation; staff reservations, walk-ins, check-in/out; invoice; admin booking and refund operations
/api/payments	Mock payment, Razorpay order/verify/webhook, guest payment history, admin payment list
/api/reviews	Create a review after check-out
/api/notifications	List, unread count, mark read
/api/services	Create and manage housekeeping, maintenance, room service, support requests
/api/hotel	Read and administer hotel settings
/api/admin	Stats, users, staff creation, user updates, action log

Database behavior

- Mongoose schemas validate categories, room status, booking/payment status, roles, and service request states.
- Booking prices are stored with the reservation (night count, subtotal, tax, total) so the confirmed quote is retained.
- Room date conflicts are checked by overlapping stay dates; pending holds older than 15 minutes no longer block availability.
- The seed script provisions a sample HotelPro Grand property, 12 rooms across four categories, and demo users.

Client development

The Vite client uses React Router for page navigation and a Redux Toolkit store for authentication state. Axios calls the API. Bootstrap 5 provides layout and controls; React Toastify displays action feedback; Recharts renders administrator charts.

Route guards hide staff and admin pages from unauthorized roles, with server-side checks enforcing access independently.

Page area	Main screens
Public	Home, room search, room details
Account and booking	Login, register, password reset, payment, my bookings, profile, notifications
Staff	Reservations, new walk-in booking, room status, housekeeping and requests
Admin	Overview, rooms and rates, bookings, users/staff, security and action log

Project Execution and Deployment

Typical end-to-end use

6. Start MongoDB and configure the API environment.
7. Start the Express API, then the Vite client. Confirm the API health endpoint at /api/health.
8. Sign in as a guest, search available rooms, select dates, and create a booking.
9. Complete the configured payment flow. In mock mode, payment is simulated; Razorpay mode requires valid keys.
10. Use a staff account to manage the reservation, check the guest in and out, and complete housekeeping.
11. Use an administrator account to review reports, manage inventory and users, and process refund requests.

Deployment outline

The supplied DEPLOY.md proposes MongoDB Atlas for data, Render for the API, and Netlify or Vercel for the client. Set the API root to server and the web build root to client. Configure production environment variables, set CLIENT_URL and VITE_API_URL to the deployed origins, and verify /api/health after deployment. Seed the production database only once and secure or disable all demo accounts.

Known limits and operational notes

- The default payment mode is mock. It is for development only and must not be treated as a real payment processor.
- Uploaded room photos are stored under server/uploads. Ephemeral hosting storage may erase them on restart; use persistent or cloud image storage for production.
- SMTP must be configured for password reset and outbound booking/invoice email delivery.
- The README recommends adding automated tests, rate limiting, Helmet, HTTPS, and production secrets/configuration before going live. This document does not claim those deployment checks have been completed.
- The repository does not provide a verified public demo URL, screenshots, demo video, or drive link; none are asserted here.

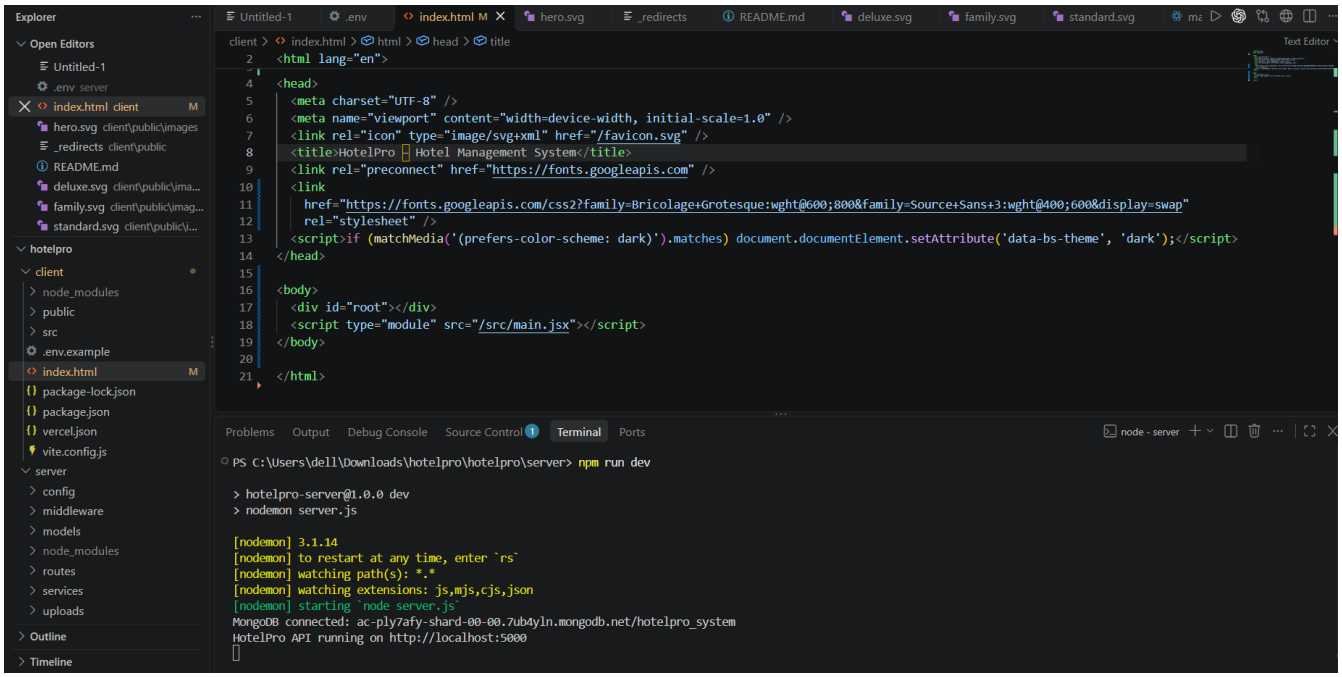
Project source

GitHub repository: **https://github.com/ChhaviJain24/HotelPro-Project**

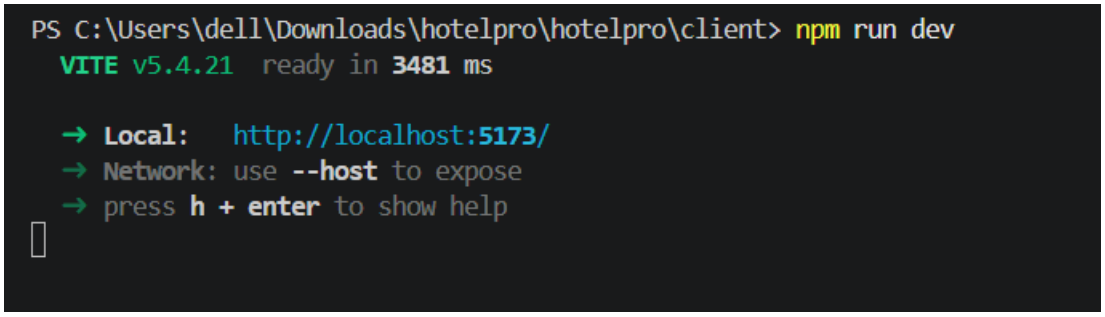
Primary implementation files include README.md, DEPLOY.md, client/src/, server/app.js, server/routes/, server/models/index.js, and server/utils/booking.js.

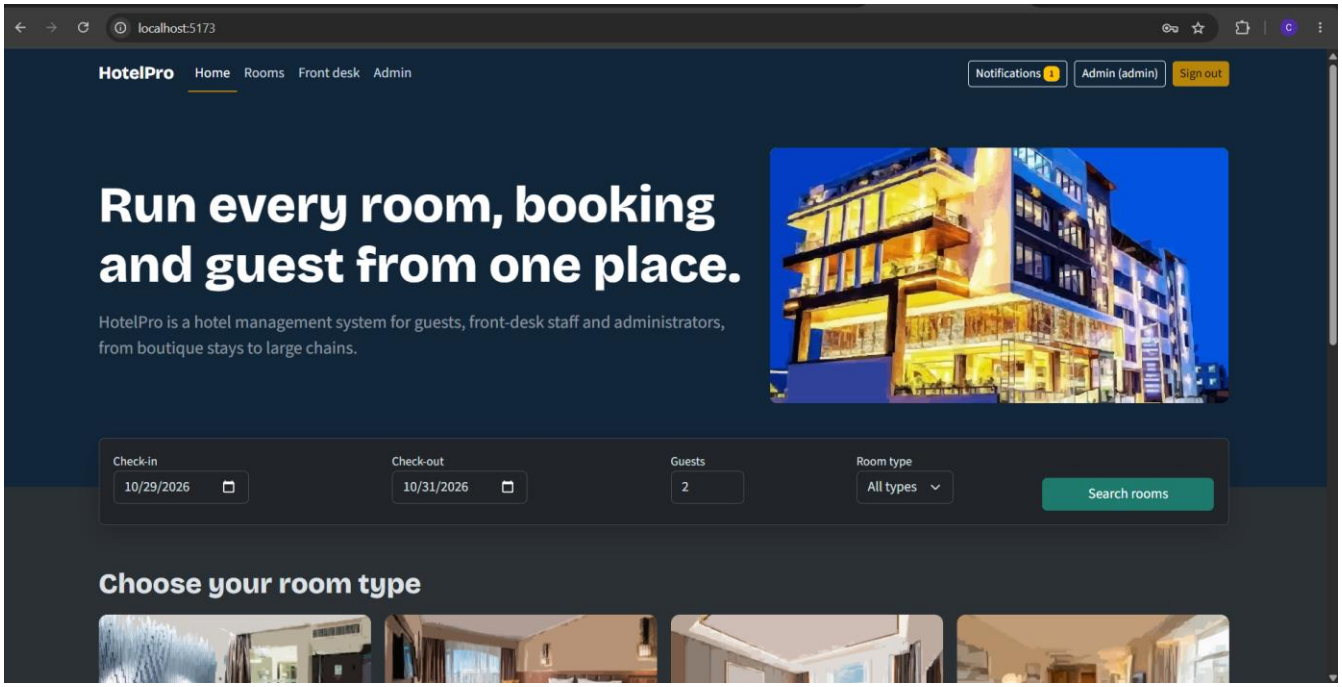
Demo vedio link - https://drive.google.com/file/d/1glDLxcT9Uzk5QKoIl9lod_02KtmTRUsH/view?usp=sharing

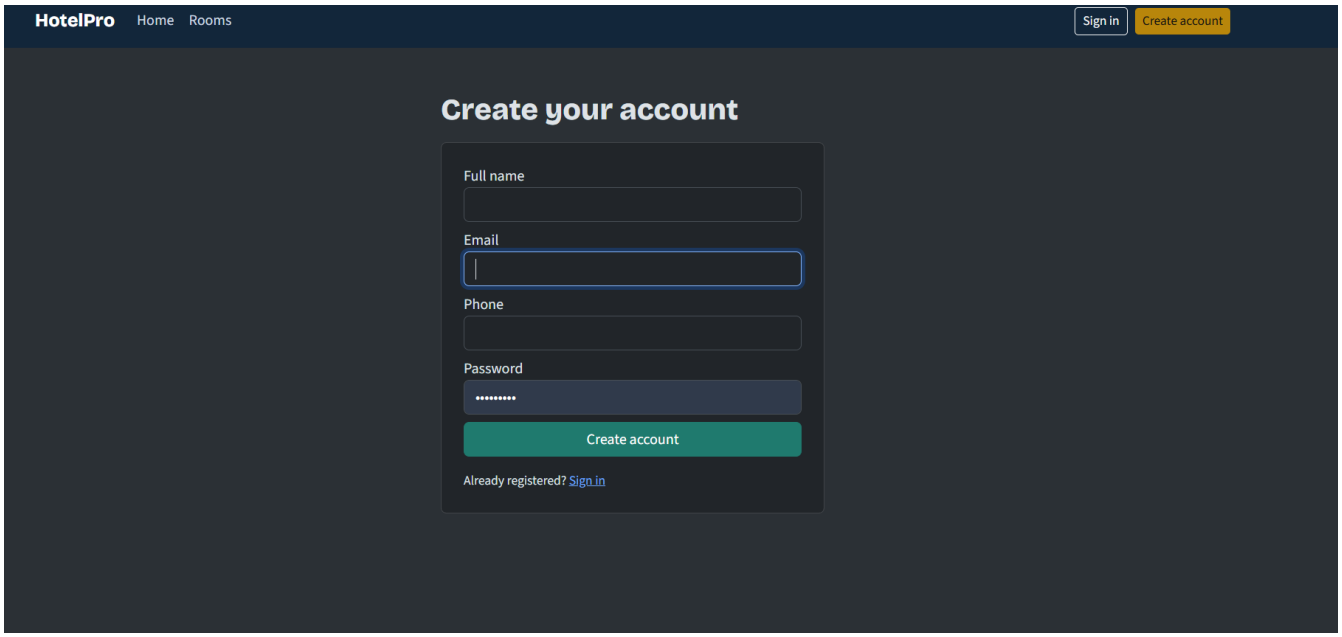
Screenshots

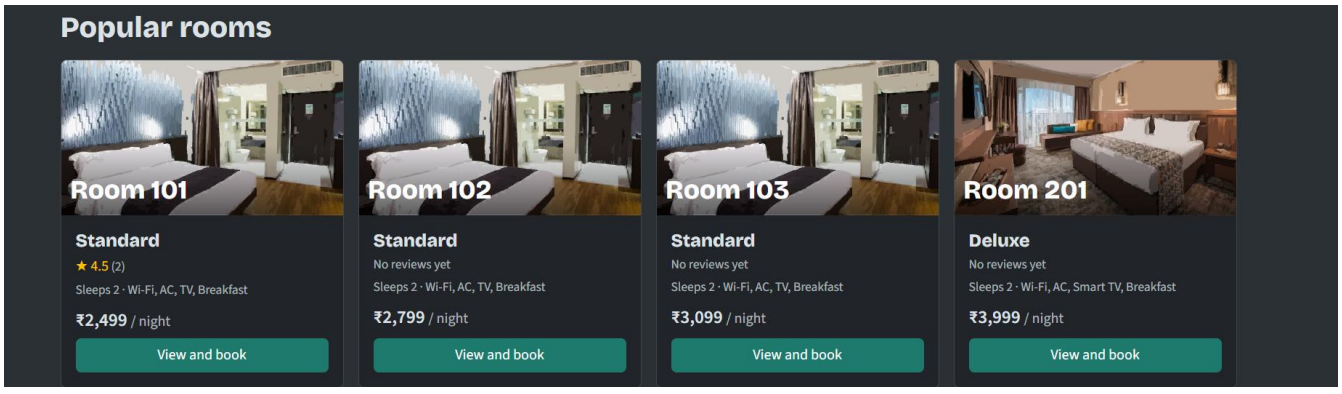


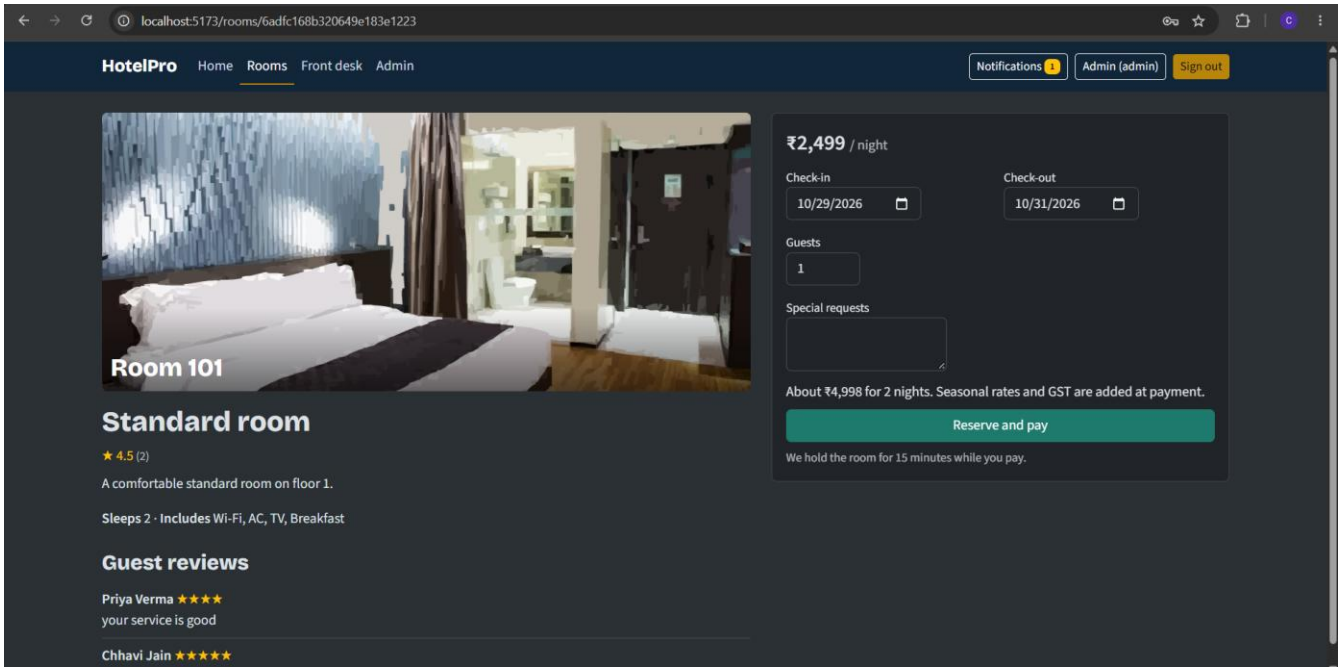


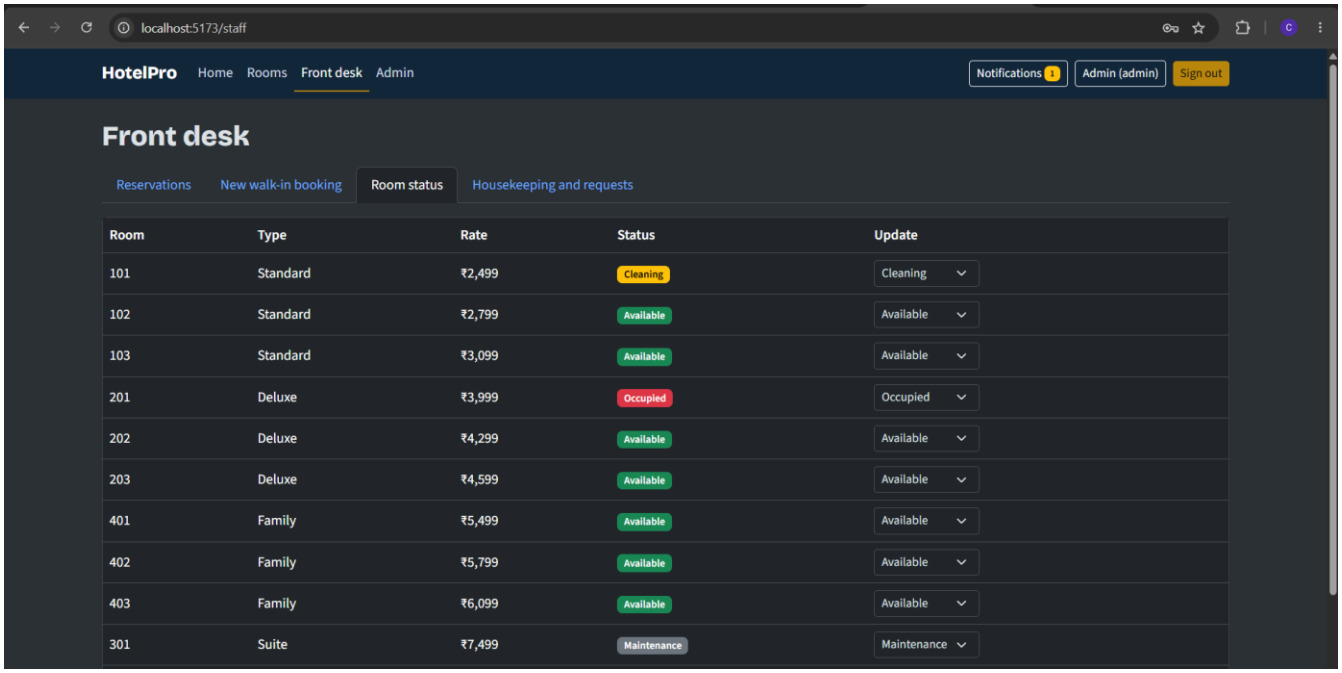


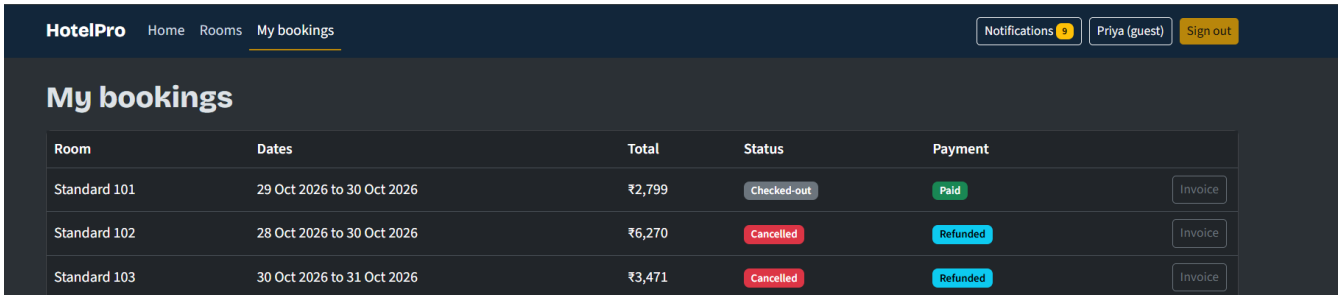

localhost:5173
HotelPro
Home
Rooms
Front desk
Admin
Notifications 1
Admin (admin)
Sign out
Run every room, booking and guest from one place.
HotelPro is a hotel management system for guests, front-desk staff and administrators, from boutique stays to large chains.
Check-in
10/29/2026
Check-out
10/31/2026
Guests
2
Room type
All types
Search rooms
Choose your room type

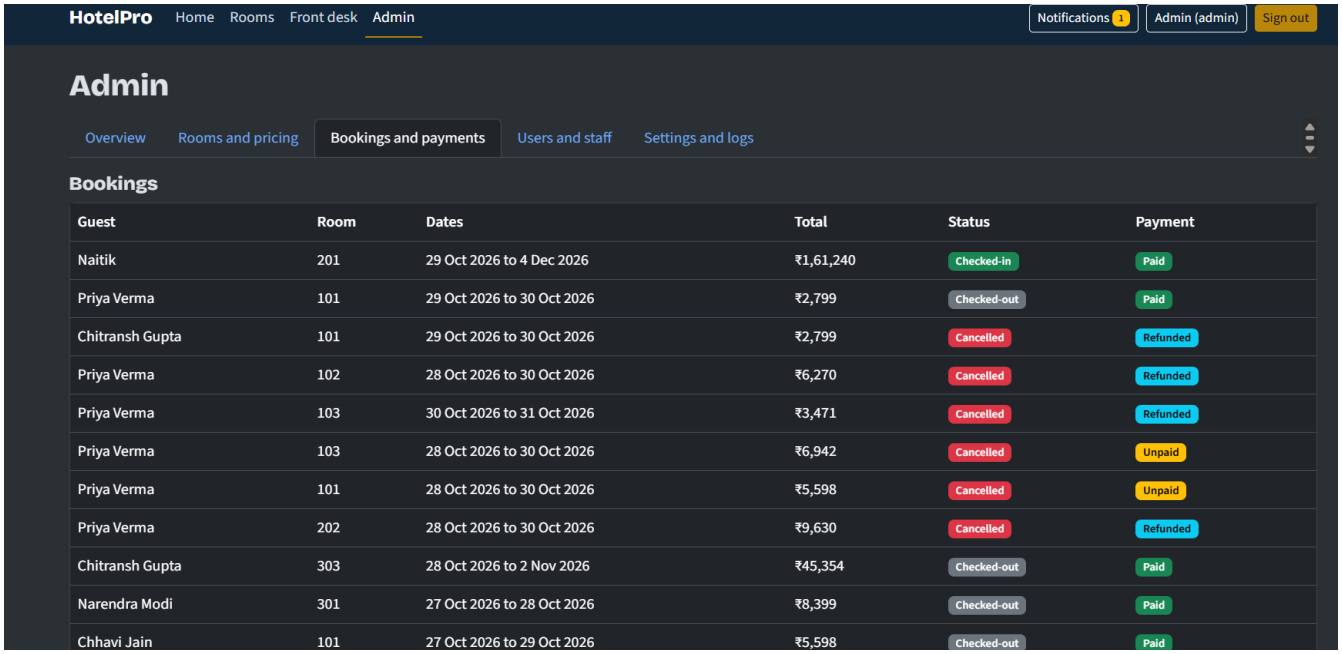

HotelPro
Home
Rooms
Sign in
Create account
Create your account
Full name
Email
Phone
Password
Create account
Already registered? Sign in

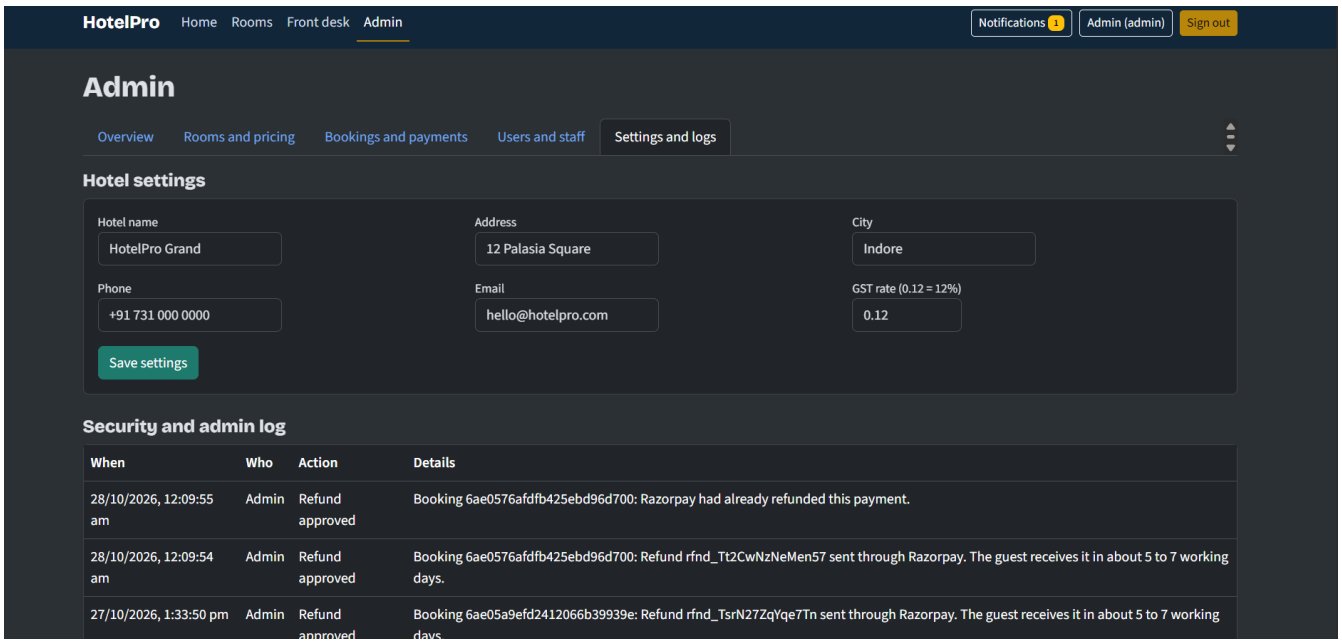

Popular rooms
Room 101
Standard
★ 4.5 (2)
Sleeps 2 · Wi-Fi, AC, TV, Breakfast
₹2,499 / night
View and book
Room 102
Standard
No reviews yet
Sleeps 2 · Wi-Fi, AC, TV, Breakfast
₹2,799 / night
View and book
Room 103
Standard
No reviews yet
Sleeps 2 · Wi-Fi, AC, TV, Breakfast
₹3,099 / night
View and book
Room 201
Deluxe
No reviews yet
Sleeps 2 · Wi-Fi, AC, Smart TV, Breakfast
₹3,999 / night
View and book

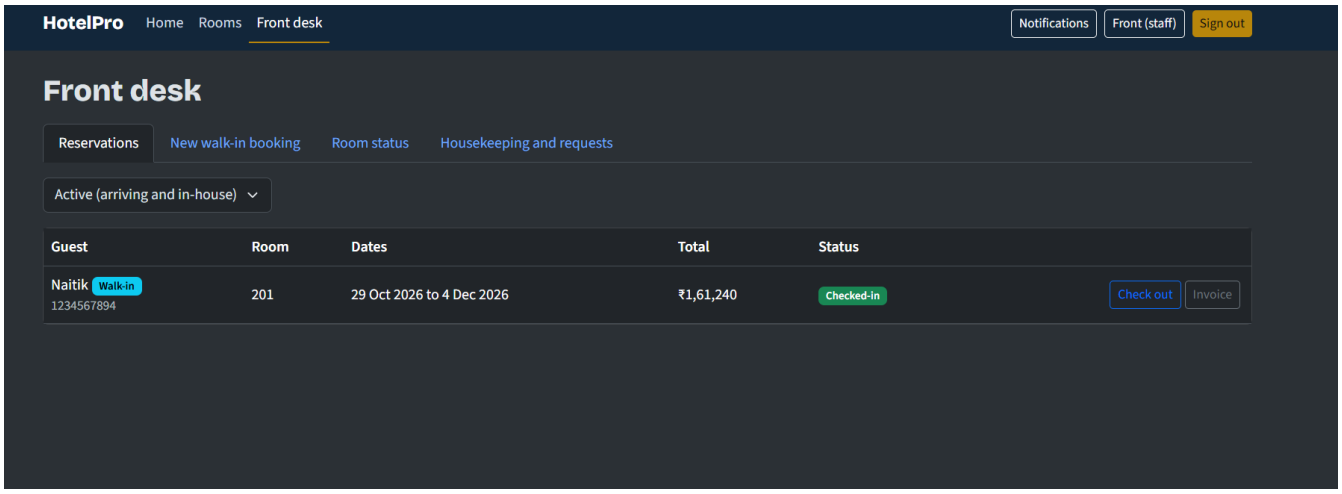

localhost:5173/rooms/6adfc168b320649e183e1223
HotelPro
Home
Rooms
Front desk
Admin
Notifications 1
Admin (admin)
Sign out
Room 101
Standard room
★ 4.5 (2)
A comfortable standard room on floor 1.
Sleeps 2 · Includes Wi-Fi, AC, TV, Breakfast
Guest reviews
Priya Verma ★★★★
your service is good
Chhavi Jain ★★★★★
₹2,499 / night
Check-in
10/29/2026
Check-out
10/31/2026
Guests
1
Special requests
About ₹4,998 for 2 nights. Seasonal rates and GST are added at payment.
Reserve and pay
We hold the room for 15 minutes while you pay.

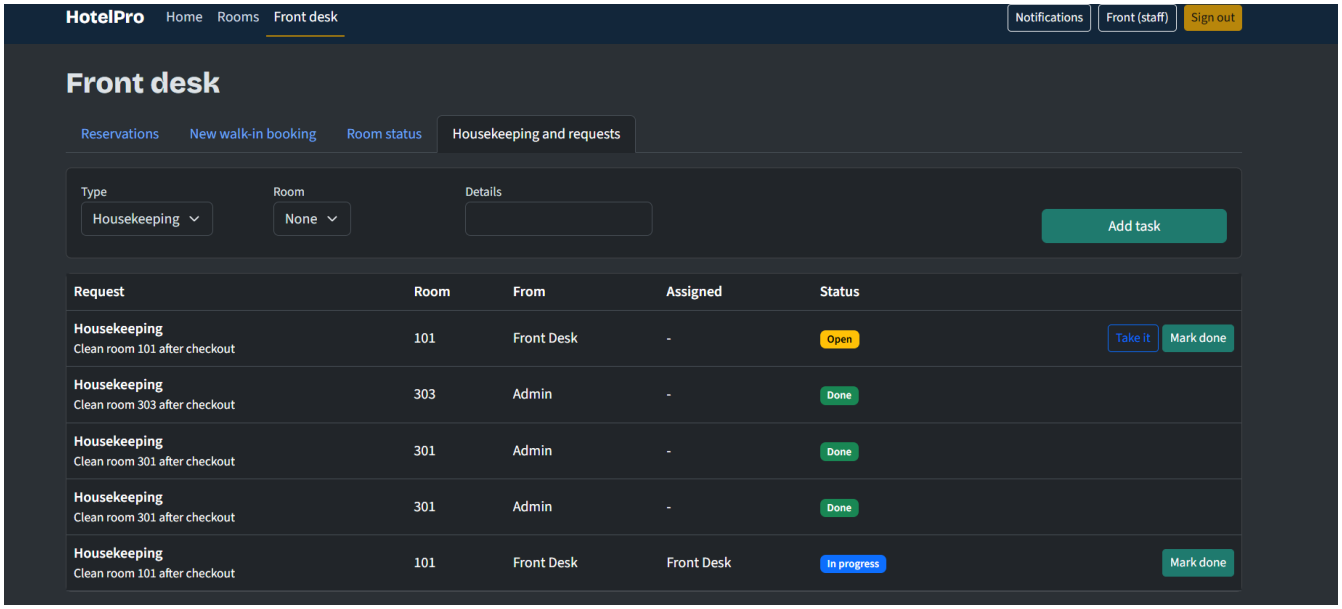

localhost:5173/staff
HotelPro
Home
Rooms
Front desk
Admin
Notifications 1
Admin (admin)
Sign out
Front desk
Reservations
New walk-in booking
Room status
Housekeeping and requests
Room
Type
Rate
Status
Update
101 Standard ₹2,499 Cleaning Cleaning
102 Standard ₹2,799 Available Available
103 Standard ₹3,099 Available Available
201 Deluxe ₹3,999 Occupied Occupied
202 Deluxe ₹4,299 Available Available
203 Deluxe ₹4,599 Available Available
401 Family ₹5,499 Available Available
402 Family ₹5,799 Available Available
403 Family ₹6,099 Available Available
301 Suite ₹7,499 Maintenance Maintenance

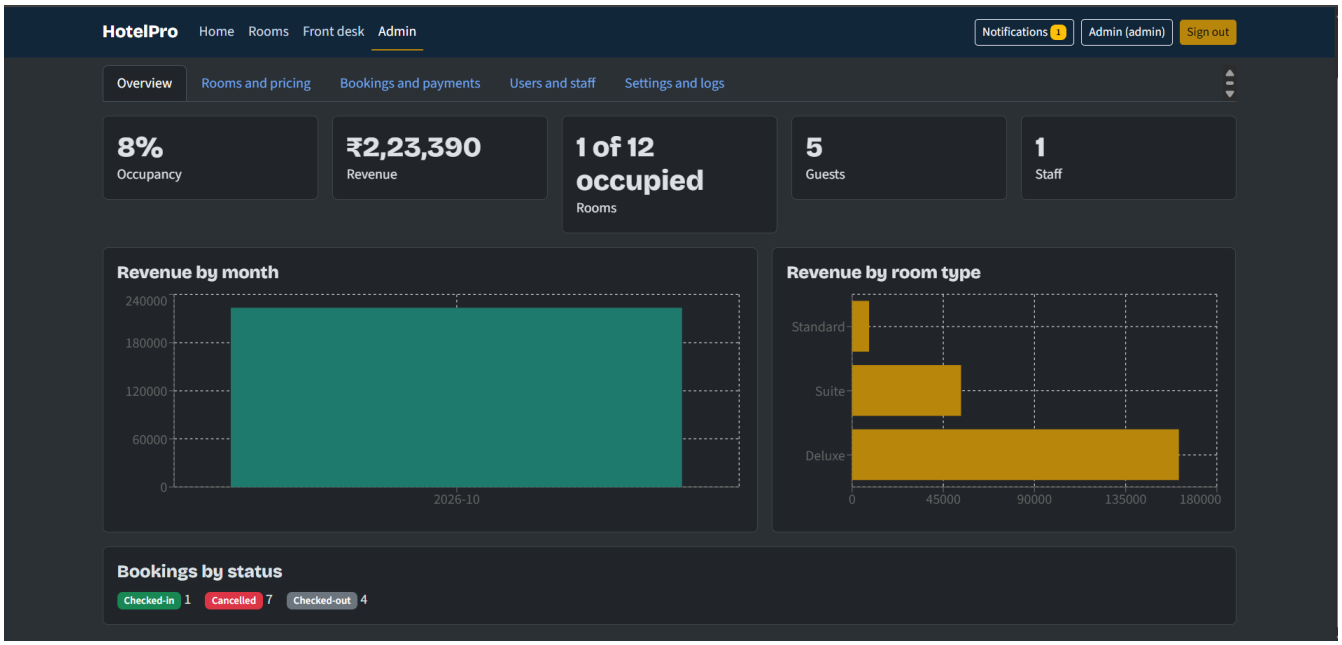

HotelPro
Home
Rooms
My bookings
Notifications 9
Priya (guest)
Sign out
My bookings
Room
Dates
Total
Status
Payment
Standard 101 29 Oct 2026 to 30 Oct 2026 ₹2,799 Checked-out Paid Invoice
Standard 102 28 Oct 2026 to 30 Oct 2026 ₹6,270 Cancelled Refunded Invoice
Standard 103 30 Oct 2026 to 31 Oct 2026 ₹3,471 Cancelled Refunded Invoice


HotelPro
Home
Rooms
Front desk
Admin
Notifications 1
Admin (admin)
Sign out
Admin
Overview
Rooms and pricing
Bookings and payments
Users and staff
Settings and logs
Bookings
Guest
Room
Dates
Total
Status
Payment
Naitik
201
29 Oct 2026 to 4 Dec 2026
₹1,61,240
Checked-in
Paid
Priya Verma
101
29 Oct 2026 to 30 Oct 2026
₹2,799
Checked-out
Paid
Chitransh Gupta
101
29 Oct 2026 to 30 Oct 2026
₹2,799
Cancelled
Refunded
Priya Verma
102
28 Oct 2026 to 30 Oct 2026
₹6,270
Cancelled
Refunded
Priya Verma
103
30 Oct 2026 to 31 Oct 2026
₹3,471
Cancelled
Refunded
Priya Verma
103
28 Oct 2026 to 30 Oct 2026
₹6,942
Cancelled
Unpaid
Priya Verma
101
28 Oct 2026 to 30 Oct 2026
₹5,598
Cancelled
Unpaid
Priya Verma
202
28 Oct 2026 to 30 Oct 2026
₹9,630
Cancelled
Refunded
Chitransh Gupta
303
28 Oct 2026 to 2 Nov 2026
₹45,354
Checked-out
Paid
Narendra Modi
301
27 Oct 2026 to 28 Oct 2026
₹8,399
Checked-out
Paid
Chhavi Jain
101
27 Oct 2026 to 29 Oct 2026
₹5,598
Checked-out
Paid


HotelPro
Home
Rooms
Front desk
Admin
Notifications 1
Admin (admin)
Sign out
Admin
Overview
Rooms and pricing
Bookings and payments
Users and staff
Settings and logs
Hotel settings
Hotel name
HotelPro Grand
Address
12 Palasia Square
City
Indore
Phone
+91 731 000 0000
Email
hello@hotelpro.com
GST rate (0.12 = 12%)
0.12
Save settings
Security and admin log
When
Who
Action
Details
28/10/2026, 12:09:55 am
Admin
Refund approved
Booking 6ae0576afdfb425ebd96d700: Razorpay had already refunded this payment.
28/10/2026, 12:09:54 am
Admin
Refund approved
Booking 6ae0576afdfb425ebd96d700: Refund rfnd_Tt2CwNzNeMen57 sent through Razorpay. The guest receives it in about 5 to 7 working days.
27/10/2026, 1:33:50 pm
Admin
Refund approved
Booking 6ae05a9efd2412066b39939e: Refund rfnd_TsrN27ZqYqe7Tn sent through Razorpay. The guest receives it in about 5 to 7 working days.


HotelPro
Home
Rooms
Front desk
Notifications
Front (staff)
Sign out
Front desk
Reservations
New walk-in booking
Room status
Housekeeping and requests
Active (arriving and in-house)
Guest
Room
Dates
Total
Status
Naitik
Walk-in
1234567894
201
29 Oct 2026 to 4 Dec 2026
₹1,61,240
Checked-in
Check out
Invoice


HotelPro
Home
Rooms
Front desk
Notifications
Front (staff)
Sign out
Front desk
Reservations
New walk-in booking
Room status
Housekeeping and requests
Type
Housekeeping
Room
None
Details
Add task
Request
Room
From
Assigned
Status
Housekeeping
Clean room 101 after checkout
101
Front Desk
-
Open
Take it
Mark done
Housekeeping
Clean room 303 after checkout
303
Admin
-
Done
Housekeeping
Clean room 301 after checkout
301
Admin
-
Done
Housekeeping
Clean room 301 after checkout
301
Admin
-
Done
Housekeeping
Clean room 101 after checkout
101
Front Desk
Front Desk
In progress
Mark done


HotelPro
Home
Rooms
Front desk
Admin
Notifications 1
Admin (admin)
Sign out
Overview
Rooms and pricing
Bookings and payments
Users and staff
Settings and logs
8%
Occupancy
₹2,23,390
Revenue
1 of 12 occupied
Rooms
5
Guests
1
Staff
Revenue by month
240000
180000
120000
60000
0
2026-10
Revenue by room type
Standard
Suite
Deluxe
0
45000
90000
135000
180000
Bookings by status
Checked-in 1
Cancelled 7
Checked-out 4